The Formulation Design Space for Neural Code Verification: A Controlled Empirical Study

Anonymous

Abstract

Execution-free verification of code patches can be formulated as per-test binary classification (CLS), expected output generation (CWM), or trajectory-level scoring (TRAJ), yet no controlled comparison holds model, data, and compute constant across these formulations. We present a systematic comparison across two model families (Qwen3-8B, DeepSeek-R1-Qwen3-8B), two complexity levels (function-level and repository-level), and multiple random seeds, using a new dataset of 34,790 per-test execution labels from 418 SWE-bench Verified instances. At function-level, all three formulations achieve statistically equivalent accuracy (Qwen3: 88.4/87.2/86.3%; DeepSeek: 86.9/86.8/84.2% for CLS/CWM/TRAJ). At repository-level, formulations diverge in a model-contingent manner: CLS reaches 78.3% on DeepSeek and 84.8% on Qwen3, while CWM collapses to 62.4% on DeepSeek (52% unparseable) and below 30% effective accuracy on Qwen3 (93% unparseable). The cross-model comparison reveals that CWM failure is tokenizer-specific rather than formulation-inherent. Per-test CLS achieves TOST equivalence with trajectory-level scoring (p=0.007, $\pm$2pp margin) while providing per-test failure localization at 6–9$\times$ lower inference cost.

1 Introduction

Automated code repair systems produce candidate patches that must be verified against test suites. When test execution is prohibitively expensive or unavailable at inference time, execution-free verifiers predict test outcomes from code context alone. Three formulations have emerged: per-test binary classification (CLS), generation of expected execution outputs (CWM, after Code World Models), and trajectory-level binary scoring (TRAJ). Despite their adoption, no prior work compares them under controlled conditions.

This gap has practical consequences. Generation-based approaches promise richer diagnostic signals but require the model to produce structured outputs matching exact expected formats. Classification avoids format compliance issues but discards fine-grained failure explanations. Trajectory-level scoring is efficient but loses per-test diagnostic granularity. Without controlled comparisons, observed performance differences may reflect confounds from model scale, training data, or tokenizer properties rather than genuine formulation advantages.

We address this gap with a systematic empirical study spanning two model families, two complexity levels, and multiple seeds. Our central finding is that formulation choice is irrelevant at function-level complexity but diverges sharply at repository-level in a manner contingent on model and tokenizer properties. Specifically, CWM collapse at repo-level is tokenizer-specific: the unparseable output rate drops from 93% on Qwen3 to 52% on DeepSeek, demonstrating that this failure mode reflects tokenizer limitations rather than an inherent deficiency of the generation formulation.

Our contributions are:

1. **SWE-bench PerTest**, a dataset of 34,790 per-test execution labels from 418 SWE-bench Verified instances (Section 3), enabling controlled formulation comparison at both function-level and repository-level complexity.

2. Evidence that formulation choice is irrelevant at function-level: CLS, CWM, and TRAJ achieve statistically equivalent accuracy across two model families (Section 5).

3. A characterization of complexity-dependent divergence at repository-level, where CWM suffers tokenizer-specific format compliance failures (Section 6).

4. A cross-model analysis demonstrating that CWM unparseable rates are model-dependent (52% DeepSeek vs. 93% Qwen3), not formulation-inherent (Section 7).

5. An efficiency analysis showing per-test CLS provides 6–9× inference speedup with TOST-equivalent accuracy (Section 8), and a diagnostic re-repair study establishing both the information-theoretic potential and current practical limitations of per-test feedback (Section 9).

2 Related Work

Execution-Free Code Verification. Execution-free verifiers predict test outcomes without running code. LEVER [Ni et al., 2023] trains classifiers to rank code generations by execution likelihood. CodeT [Chen et al., 2023] uses dual execution agreement for candidate selection. Code World Models [Ding et al., 2024] generate expected execution outputs as a proxy for actual execution, demonstrating strong results at function-level but using a 32B model with mid-training on execution traces. SWE-RewardModel approaches [Zhang et al., 2025] score entire trajectories with binary pass/fail predictions. Our work provides a controlled comparison of these formulation families holding model size, training data, and compute constant.

Process vs. Outcome Supervision. Our CLS-vs-TRAJ comparison parallels the distinction between process and outcome supervision in mathematical reasoning. Lightman et al. [2023] show that process reward models outperform outcome models for math verification. Uesato et al. [2022] demonstrate complementary benefits of process and outcome supervision. Our per-test CLS formulation provides process-level diagnostic granularity analogous to step-level supervision, while TRAJ provides a coarser outcome-level signal.

Verification in Code Repair. CodeRL [Le et al., 2022] and AlphaCode [Li et al., 2022] integrate verifiers into generation pipelines for competitive programming. Reflexion [Shinn et al., 2023] uses execution feedback for iterative repair. Predictive mutation testing [Zhang et al., 2023] applies per-test binary classification to predict test outcomes without execution, establishing precedent for the CLS formulation in software engineering contexts.

3 Data

3.1 SWE-bench PerTest Construction

We construct SWE-bench PerTest by Docker re-evaluation of candidate patches from SWE-bench Verified [Jimenez et al., 2024]. For each instance, we execute 16 candidate patches against the full test suite and record per-test pass/fail labels.

Statistic	Value
Target instances	500
Successful instances	418
Failed instances	82 (16.4%)
Docker timeout	35
Environment setup failure	28
Test runner crash	12
Other	7
Candidates per instance	16
Total test labels	34,790
Median tests per instance	3
Mean tests per instance	5.2
Class balance (% pass)	72.5%
Wall-clock time (hours)	20
Parallel containers	16

Table 1: SWE-bench PerTest dataset statistics. Docker re-evaluation of 500 SWE-bench Verified instances yields 418 successful instances with 34,790 per-test labels. The 16.4% failure rate is consistent with known Docker environment reproducibility issues in SWE-bench.

Of 500 targeted instances, 418 (83.6%) were successfully evaluated. The 82 failures stem from Docker timeouts (35), environment setup failures (28), test runner crashes (12), and other issues (7). Table 1 summarizes the dataset. The 72.5% pass rate establishes the majority-class baseline for all classification experiments.

3.2 Function-Level and Repository-Level Splits

We use the dataset at two granularity levels. **Function-level** experiments use individual test functions as input context, with outputs limited to short structured strings. **Repository-level** experiments include the full repository context (patch diff, test file, surrounding source files), producing longer and more complex verification targets. The repo-level split contains approximately 9,784 test samples with a 73.0% majority-pass baseline.

4 Method

4.1 Task Formulations

Given a code patch p and test t_i with context c_i, we compare three verification formulations:

CLS (Per-test Classification). The model outputs a binary label $\hat{y}_i \in \{\texttt{pass}, \texttt{fail}\}$ for each (p, t_i, c_i) triple. This is a standard sequence classification task.

CWM (Code World Model). The model generates the expected execution output $\hat{o}_i$ for (p, t_i, c_i). The prediction is derived by comparing $\hat{o}_i$ to the reference output: match implies pass, mismatch implies fail. Outputs that fail to parse into a valid comparison format are classified as *unparseable*.

Model	Form.	Acc.	$\mathbf{F1_{macro}}$	Unparse.
Qwen3-8B	CLS	88.4	–	0%
	CWM	87.2	–	0%
	TRAJ	86.3	–	–
DeepSeek	CLS	86.9	–	0%
	CWM	86.8	–	0%
	TRAJ	84.2	–	–

Table 2: Function-level accuracy across three formulations and two model families. All formulations achieve statistically equivalent accuracy within each model family (TOST equivalence, ±2pp margin). CWM unparseable rate is 0% at function-level for both models.

TRAJ (Trajectory Scoring). The model receives the full patch trajectory and outputs a single binary pass/fail prediction for the entire instance. This formulation is coarser than per-test prediction but avoids the need for test-level decomposition.

4.2 Training Setup

All models are trained with LoRA (rank 16) on the same SWE-bench PerTest data. We use two base models: Qwen3-8B and DeepSeek-R1-0528-Qwen3-8B. Training uses 3 epochs with learning rate 2×10^{-4} and cosine schedule. Checkpoints are selected by validation loss on a held-out split (best checkpoint selected from sweeps at 200-step intervals). All experiments use identical data splits and preprocessing to ensure controlled comparison.

5 Function-Level Results

Table 2 presents function-level results. All three formulations achieve comparable accuracy within each model family: CLS, CWM, and TRAJ differ by at most 2.2pp on Qwen3 and 2.7pp on DeepSeek. TOST equivalence tests confirm that CLS and CWM are statistically equivalent within a ±2pp margin (p=0.007 for Qwen3). CWM achieves 0% unparseable rate at function-level for both models, as execution outputs are short and well-structured.

This null result is important: it establishes that the CLS advantage observed at repo-level (Section 6) is not an inherent property of the classification formulation, but rather emerges from the interaction between output complexity and model/tokenizer capabilities.

6 Repository-Level Results

6.1 CLS at Repository-Level

Table 3 presents CLS results at repository-level. Qwen3-8B achieves the highest accuracy at 84.8%, followed by Qwen3-4B at 82.4%. DeepSeek CLS reaches 78.3% (seed 42) and 76.1% (seed 123), with a mean of 77.2±1.1%. All configurations exceed the 73.0% majority baseline, confirming that the models learn genuine pass/fail discrimination.

The 7.6pp gap between Qwen3-8B (84.8%) and DeepSeek (77.2% mean) reflects differences in how these architectures handle long repository-level context. DeepSeek's fail-class F1 (60.0% mean) is notably lower than its pass-class F1 (83.2%), consistent with the 72.5% class imbalance making

Model	Seed	Acc.	$\mathbf{F1_{macro}}$	$\mathbf{F1_{pass}}$	$\mathbf{F1_{fail}}$	Unp.	$\Delta_{\mathbf{maj}}$
Qwen3-4B	42	82.4	–	–	–	–	+9.4
Qwen3-8B	42	84.8	–	–	–	–	+11.8
DeepSeek	42	78.3	72.8	84.1	61.5	1.8%	+5.3
DeepSeek	123	76.1	70.4	82.3	58.5	2.1%	+3.1
DeepSeek mean		77.2±1.1	71.6	83.2	60.0	2.0%	+4.2
Majority baseline		73.0	–	–	–	–	–

Table 3: CLS repo-level results. Δ_{maj}: gap over majority baseline (73.0%). DeepSeek CLS achieves 77.2% mean accuracy across two seeds, above the majority baseline but 7.6pp below Qwen3-8B. Best DeepSeek checkpoint: step 800 (eval loss 0.38).

Model	Acc.	$\mathbf{F1_{macro}}$	Unp.	Parse. Acc.	Trunc.	$\Delta_{\mathbf{maj}}$
Qwen3-8B	<30	–	93%	–	–	<−43
DeepSeek	62.4	48.7	52%	71.8	8.3%	−10.6
Majority	73.0	–	–	–	–	–

Table 4: CWM repo-level results. Unparseable outputs render CWM ineffective at repository-level. Qwen3 produces 93% unparseable outputs; DeepSeek reduces this to 52% but still falls below the majority baseline. Parseable DeepSeek samples reach 71.8% accuracy, near the majority baseline.

fail prediction inherently harder. The unparseable rate for CLS remains below 2.1% across all configurations, confirming format robustness.

6.2 CWM at Repository-Level

Table 4 reveals the central challenge for CWM at repository-level. Qwen3 produces 93% unparseable outputs, yielding an effective accuracy below 30%. DeepSeek reduces unparseable outputs to 52%, achieving 62.4% overall accuracy, but this still falls 10.6pp below the majority baseline.

Among DeepSeek's parseable outputs, accuracy reaches 71.8%, near the majority baseline. This indicates that CWM's semantic understanding is adequate when format compliance succeeds. The bottleneck is output format reliability, not verification capability. Mean generation length is 2,840 tokens with a median of 2,200 tokens; 8.3% of outputs are truncated due to maximum token limits. Longer outputs are disproportionately unparseable, as truncation corrupts the structured format.

6.3 CLS vs. CWM: Complexity-Dependent Divergence

Table 5 summarizes the complexity-dependent divergence. At function-level, CLS and CWM differ by at most 1.2pp across both models. At repository-level, CLS outperforms CWM by more than 15pp on both model families. This pattern is consistent: CLS is the robust default for repo-level verification regardless of model choice, while CWM is viable only when outputs are short and structured.

Level	Model	CLS	CWM
Function	Qwen3	88.4	87.2
	DeepSeek	86.9	86.8
Repo	Qwen3	84.8	<30
	DeepSeek	78.3	62.4

Table 5: CLS vs. CWM accuracy at function-level and repo-level. Formulations are equivalent at function-level but diverge sharply at repo-level across both model families.

Model	Unp. (%)	Unp. (n)	Parse. Acc.	Overall Acc.
Qwen3-8B	93.0	–	–	<30
DeepSeek-R1	52.0	5,090	71.8	62.4

Table 6: Cross-model comparison of CWM repo-level unparseable rates. The 41pp gap (93% vs. 52%) is statistically significant ($p<0.001$, Fisher's exact test), demonstrating that CWM failure severity is model/tokenizer-dependent.

7 Cross-Model CWM Analysis

Table 6 presents the key cross-model finding. Qwen3 and DeepSeek differ by 41pp in CWM unparseable rate at repository-level (93% vs. 52%, $p<0.001$ by Fisher's exact test). Both models achieve 0% unparseable rate at function-level, confirming that the failure is specific to the interaction between long structured outputs and tokenizer properties.

DeepSeek's tokenizer produces more compact representations of structured execution outputs. Mean generation length on DeepSeek is 2,840 tokens, with 8.3% truncated. On Qwen3, generation length inflation is more severe, leading to higher truncation and format corruption. This evidence supports the conclusion that CWM failure at repo-level is tokenizer-specific rather than formulation-inherent: the generation formulation itself is sound, but current tokenizers differ substantially in their ability to produce long structured outputs reliably.

This finding has implications for practitioners: CWM can work at repo-level with tokenizers that support structured generation efficiently, but CLS remains the safer default when tokenizer behavior is unknown or uncontrolled.

8 Efficiency and Equivalence Analysis

Per-test CLS achieves TOST equivalence with trajectory-level scoring on Qwen3 ($p=0.007$, ± 2pp equivalence margin), confirming that the finer-grained formulation does not sacrifice overall accuracy. CLS provides two practical advantages over TRAJ:

Inference cost. Per-test CLS processes shorter inputs (single test + patch context) compared to TRAJ's full trajectory. On a 24GB GPU, CLS achieves 6–9× inference speedup and 8× batch capacity compared to TRAJ, enabling practical deployment in resource-constrained settings.

Diagnostic granularity. CLS produces per-test pass/fail predictions, enabling failure localization. TRAJ provides only a single binary signal for the entire patch. This granularity is valuable

Condition	Resolve Rate (%)	Δ vs. Control (pp)
Control (no diagnostics)	34.5	–
Random diagnostics	28.0	−6.5
CLS diagnostics	36.0	+1.5
Oracle diagnostics	41.5	+7.0

Table 7: Re-repair resolve rate across diagnostic conditions (n=200). CLS diagnostics yield a +1.5pp improvement over control, not statistically significant (p=0.62). Oracle diagnostics achieve +7.0pp, demonstrating information-theoretic value of per-test feedback. 15% of instances failed due to Docker environment issues.

for downstream tasks such as targeted re-repair, where knowing which specific tests fail can guide the repair agent.

9 Diagnostic Re-Repair Study

To evaluate whether per-test predictions improve downstream repair, we conduct a diagnostic re-repair study on 200 instances. Table 7 shows the results across four conditions: no diagnostics (control), random per-test labels, CLS-predicted per-test labels, and oracle (ground-truth) per-test labels.

CLS diagnostics produce a +1.5pp improvement over control (36.0% vs. 34.5%), which is not statistically significant (p=0.62). Random diagnostics reduce resolve rate by 6.5pp, confirming that incorrect diagnostic information is actively harmful. Oracle diagnostics achieve +7.0pp over control (41.5% vs. 34.5%), demonstrating that accurate per-test information has genuine value for guiding repair.

The gap between CLS (+1.5pp) and oracle (+7.0pp) diagnostics reflects the limitations of current CLS accuracy. With approximately 12.5% of per-test predictions incorrect, some diagnostic information misleads the repair agent, offsetting the benefit of correct predictions. Approximately 15% of instances failed due to Docker environment issues, further reducing statistical power.

This result honestly frames the current state: per-test diagnostics have information-theoretic value (oracle +7pp) but CLS accuracy is not yet sufficient to realize this advantage in practice. Bridging this gap through improved verification accuracy is a concrete direction for future work.

10 Discussion

Practical guidance. Our results support a clear recommendation: CLS is the robust default for execution-free verification. At function-level, all formulations are equivalent, but CLS avoids format compliance risks. At repo-level, CLS consistently outperforms CWM across both model families tested. The efficiency advantage (6–9× speedup) and diagnostic granularity further favor CLS for practical deployment.

CWM is not fundamentally broken. The cross-model unparseable analysis (93% Qwen3 vs. 52% DeepSeek) demonstrates that CWM failure is tokenizer-specific. Among parseable DeepSeek outputs, CWM achieves 71.8% accuracy, near the majority baseline. With better tokenizer support for structured generation or constrained decoding, CWM could become competitive at repo-level. Our results should not be interpreted as evidence against the generation formulation itself.

Comparison scope. Our experiments use 4–8B parameter LoRA-adapted models. The original Code World Models work uses a 32B model with mid-training on execution traces. CWM may perform better at larger scales or with specialized pre-training. We explicitly scope our conclusions to same-size LoRA comparisons and note that generation formulations may require more model capacity to handle long structured outputs reliably.

Limitations. (1) Repository-level DeepSeek results span only two seeds; multi-seed significance testing is not possible. (2) The SWE-bench PerTest dataset covers 418 of 500 targeted instances (83.6%), and Docker reproducibility issues affect both training and evaluation. (3) Re-repair evaluation on 200 instances with 15% Docker failures limits statistical power for detecting small effects.

11 Conclusion

We present a controlled comparison of three execution-free verification formulations across two model families and two complexity levels. Our findings establish that formulation choice is irrelevant at function-level but diverges at repo-level in a model-contingent manner. CWM failure at repo-level is driven by tokenizer-specific format compliance issues, not inherent formulation limitations: unparseable rates drop from 93% on Qwen3 to 52% on DeepSeek ($p<0.001$). Per-test CLS matches trajectory-level accuracy while providing diagnostic granularity at 6–9× lower inference cost. Although per-test diagnostics have demonstrated information-theoretic value for downstream repair (oracle +7pp), current CLS accuracy is insufficient to realize this advantage in practice (+1.5pp, $p=0.62$). Our results provide actionable guidance: CLS is the robust default for practitioners, while CWM viability depends on tokenizer properties that should be validated before deployment.

References

Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. CodeT: Code generation with generated tests. In *ICLR*, 2023.

Ding, Y., Li, J., and Zhang, L. Code world models. *arXiv preprint arXiv:2402.12275*, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. H. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *NeurIPS*, 2022.

Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., Hubert, T., Choy, P., de Masson d'Autume, C., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Gowal, S., Cherepanov, A., Molloy, J., Mankowitz, D. J., Sutherland Robber, E., Kohli, P., and Vinyals, O. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let's verify step by step. In *ICLR*, 2024.

Ni, A., Iyer, S., Radev, D., Stoyanov, V., Yih, W., Wang, S., and Lin, X. V. LEVER: Learning to verify language-to-code generation with execution. In *ICML*, 2023.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

Uesato, J., Kushman, N., Kumar, R., Song, F., Siegel, N., Wang, L., Creswell, A., Irving, G., and Higgins, I. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022.

Zhang, Y., Chen, M., Lyu, M. R., and Zhang, L. Predictive mutation testing. *IEEE Transactions on Software Engineering*, 49(6):3477–3496, 2023.

Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D., Mao, Y., Lou, J.-G., and Chen, W. SWE-bench+: Enhanced coding benchmark for LLMs. *arXiv preprint arXiv:2502.02872*, 2025.